

SUPPLEMENTARY MATERIALS

Supplementary Methods and Experimental Details

1. Peptide graph construction features

To construct peptide graphs, we designed both node-level and edge-level features to encode sequence-derived and physicochemical information. The node features included Kyte–Doolittle hydrophathy, fractional side-chain charge, local hydrophathy difference, one-hot encoding, BLOSUM62 encoding, and Z-scale encoding. These features were used to represent residue-specific physicochemical properties and sequence identity information. A summary of the node features used for peptide graph construction is provided in **Table S1**.

Edge features were designed to describe the relationships between residue pairs in the peptide graph. Specifically, sequence distance, hydrophathy product, and side-chain charge product were used to encode positional proximity, hydrophobic similarity, and electrostatic compatibility between connected residues. A summary of the edge features is provided in **Table S2**.

2. Model training and hyperparameter settings

To ensure reliable predictive performance, DeepCPP was trained for 50 epochs with a batch size of 64. The model was optimized using AdamW with an initial learning rate of 0.001 and a weight decay of 0.0001. A ReduceLROnPlateau learning-rate scheduler was used to adaptively reduce the learning rate when the validation score reached a plateau. The peptide graphs were constructed using a hybrid topology that combines local sequence-window edges (window = 3) and physicochemical KNN edges ($k = 8$), with self-loops included. The GLGA graph branch used a hidden dimension of 128 and a subgraph selection ratio of 0.3, while both the ESM-2 branch and the graph branch were projected to 64-dimensional branch embeddings before fusion. ESM-2 sequence representations were generated from a pretrained ESM-2 checkpoint using mean and max pooling over residue-level embeddings.

A coarse hyperparameter search was conducted to select the final training configuration. The searched ranges and final selected values are summarized in **Table S3**.

The HSIC regularization weight was searched over $\{0, 0.1, 0.3, 0.5, 0.7, 1.0\}$, and 0.5 was selected as the final value because it provided a favorable balance between reducing cross-branch redundancy and maintaining predictive performance. DeepCPP uses symmetric pairwise gated cross-attention between the two branch-level embeddings rather than conventional multi-head attention; therefore, the number of attention heads is not a tunable hyperparameter in this implementation.

The ESM-2 parameters were initialized from the pretrained esm2_t6_8M_UR50D checkpoint [1]. Our experimental development was based on Python [2], and we used the PyTorch framework to construct and train the model [3].

Supplementary Results

1. Overview of the CPP and Non-CPP data

To explore the differences between CPPs and non-CPPs, we conducted a thorough analysis of their amino acid compositions and sequence lengths. **Figure S1A** illustrates the differences in amino acid composition (AAC) between CPPs and non-CPPs. The analysis revealed that CPPs are enriched in basic residues, most notably arginine (R) and, to a lesser extent, lysine (K), which aligns with their cationic nature and electrostatic interactions with anionic membranes. Non-CPPs show higher proportions of acidic residues aspartic acid (D) and glutamic acid (E), together with valine (V), indicating a lower net positive charge.

Figure S1B compares the sequence length distributions of CPPs and non-CPPs. Both groups consist primarily of short peptides. CPPs peak around 10-20 residues with a modest long tail, whereas non-CPPs exhibit shoulder peaks in the 10-20 and 35-40 residue ranges.

2. Sensitivity analysis of the GLGA subgraph selection ratio

To investigate the effect of the GLGA subgraph selection ratio, we evaluated ratio values from 0.1 to 0.9 under the same data split, feature construction strategy, training hyperparameters, and overall model framework. Only the Top-k subgraph selection ratio in the GLGA module was varied.

As summarized in **Table S4**, the model achieved its best overall performance at a ratio of 0.3, with the highest accuracy, sensitivity, specificity, MCC, and AUC. The results indicate that intermediate subgraph ratios are more suitable for peptide graph representation learning. Retaining too few residues may discard informative local motifs, whereas retaining too many residues may weaken the focusing effect of subgraph selection. Therefore, a ratio of 0.3 was selected as the default setting for DeepCPP.

3. Ablation experiments

3.1. Ablation analysis of alternative graph aggregation or readout strategies

To clarify how GLGA differs from standard graph-level aggregation strategies, we compared it with mean pooling, max pooling, global attention pooling, TopKPool, SAGPool, and Graph Transformer under identical experimental settings.

As shown in **Table S5**, GLGA achieved the best performance across all evaluated metrics, including accuracy, sensitivity, specificity, MCC, and AUC. These results support the global-local design of GLGA. Unlike conventional pooling methods that mainly compress node representations into a graph-level vector, GLGA first identifies salient subgraph information, refines it through local message passing, and then broadcasts the resulting subgraph context back to the full graph through a learned gate. This design helps preserve both local peptide motifs and broader physicochemical context.

3.2. Ablation analysis of fusion strategy

To evaluate whether the proposed fusion design provides an advantage over conventional multimodal integration methods, we compared cross-attention fusion with concatenation, late

fusion, gated fusion, and bilinear fusion under the same training and evaluation protocol. Concatenation directly combines the branch embeddings, late fusion averages branch-level predictions, gated fusion adaptively weights modality representations, and bilinear fusion models second-order interactions between modalities.

As shown in **Table S6**, cross-attention fusion achieved the best performance across all metrics. This suggests that simple concatenation, decision-level averaging, gated weighting, or bilinear interaction is insufficient to fully capture the complementarity between ESM-2 sequence representations and peptide graph representations. The cross-attention strategy more effectively models cross-branch dependencies while preserving modality-specific information.

3.3 Ablation analysis of model variants

To assess the role of each component in DeepCPP, we performed a component-wise ablation under identical training and evaluation settings; results are shown in **Figure S2A-B**. Using only the protein large-language-model branch (ESM+MLP) or only the multi-view residue-graph representation (GLGA+MLP) underperforms the full model, indicating the complementarity between contextual sequence embeddings and physicochemical residue-graph cues. Removing the focusing mechanism (w/o Top-k pooling) keeps the ROC competitive but degrades threshold-dependent classification performance, implying that Top-k sharpens predictions by emphasizing high-confidence substructures. Disabling the subgraph-to-graph gated broadcast (w/o Graph gated broadcast) yields a moderate decline, supporting its role in integrating local and long-range information. Eliminating the decorrelation regularizer (w/o HSIC) weakens thresholded metrics, suggesting that maintaining statistical independence across branches improves separability. Dropping cross-branch interaction (w/o Cross-attention) causes the most pronounced loss in sensitivity, underscoring the importance of gated cross-attention for aligning and fusing the two representations. Replacing the spline-based head with a linear map (w/o KAN layer) leads to a small decrease in most overall metrics, indicating that the KAN's nonlinear functional expansion provides stable gains near the decision margin. Overall, DeepCPP achieves the strongest performance and dominates the ROC in the mid-to-high recall region, indicating that these gains arise from the coordinated interplay of the model's constituent modules rather than reliance on any single part.

4. Baseline comparisons

4.1. Comparison with machine-learning baseline models

To better position the contribution of DeepCPP, we compared it with classical machine-learning classifiers using handcrafted peptide descriptors [4,5]. The machine-learning baselines included Random Forest, Extra Trees, Gradient Boosting, AdaBoost, and K-Nearest Neighbors.

As summarized in **Table S7**, DeepCPP achieved the best overall performance among all machine-learning baseline models, with the highest accuracy, sensitivity, MCC, and AUC. Although Random Forest achieved higher specificity, its lower sensitivity and MCC indicate less balanced classification performance. These results suggest that the proposed multi-view graph and protein language model fusion framework provides more robust discrimination than classical

machine-learning models based on handcrafted peptide descriptors.

4.2. Comparison with deep-learning baseline models

We further compared DeepCPP with mainstream deep-learning baseline models using one-hot encoded peptide sequences [6,7]. The deep-learning baselines included CNN, TCN, Transformer, and MLP models.

As summarized in **Table S8**, DeepCPP achieved the best overall performance among all deep-learning baseline models, with the highest accuracy, sensitivity, MCC, and AUC. Although Transformer achieved higher specificity, its lower sensitivity and MCC indicate less balanced classification performance. These results demonstrate that integrating ESM-2 sequence representations with peptide graph representations provides more robust discrimination than conventional sequence-only neural networks.

References

- [1] Lin, Zeming, et al. "Evolutionary-scale prediction of atomic-level protein structure with a language model." *Science* 379.6637 (2023): 1123-1130.
- [2] Van Rossum, Guido, and Fred L. Drake Jr. *Python tutorial*. Vol. 620. Amsterdam, The Netherlands: Centrum voor Wiskunde en Informatica, 1995.
- [3] Paszke, Adam, et al. "Automatic differentiation in pytorch." (2017).
- [4] Zhen Chen, Xuhan Liu, Pei Zhao, Chen Li, Yanan Wang, Fuyi Li, Tatsuya Akutsu, Chris Bain, Robin B Gasser, Junzhou Li, et al. ifeatureomega: an integrative platform for engineering, visualization and analysis of features from molecular sequences, structural and ligand datasets. *Nucleic acids research*, 50(W1):W434–W447, 2022.
- [5] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.
- [6] Colin Lea, Michael D Flynn, Rene Vidal, Austin Reiter, and Gregory D Hager. Temporal convolutional networks for action segmentation and detection. In *proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 156–165, 2017.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Supplementary Tables

Table S1. Summary of node features for peptide graph construction.

Node features	Dimension	Description
Kyte–Doolittle hydropathy (KD)	1	Per-residue hydropathy scale value
Fractional side-chain charge (SC_charge _i)	1	Fractional side-chain charge computed via the Henderson–Hasselbalch relation using model-compound pKa
Local hydropathy difference (Δ KD)	1	First-order difference of hydropathy along sequence
One-hot encoding	20	One-hot encoding over the 20 standard residues (ARNDCQEGHILKMFPSTWYV)
BLOSUM62 encoding	20	Per-position 20D BLOSUM62 vector
Z-scale encoding	5	Five-dimensional physicochemical descriptor vector

Table S2. Summary of edge features for peptide graph construction.

Edge features	Dimension	Description
Sequence distance $ i-j $	1	Absolute difference of residue indices
Hydropathy product $KD_i \times KD_j$	1	Product of endpoint hydropathy values to encode hydrophobic similarity
Side-chain charge product $SC_charge_i \times SC_charge_j$	1	Product of endpoint fractional charges to encode electrostatic compatibility

Table S3. Summary of hyperparameter search ranges and final training settings for DeepCPP.

Hyperparameter	Candidate range	Final value
Optimizer	Adam, AdamW	AdamW
Learning rate	1e-4, 3e-4, 1e-3, 3e-3	0.001
Weight decay	0, 1e-5, 1e-4, 1e-3	0.0001
Batch size	32, 64, 128	64
Training epochs	30, 50, 80	50
Graph hidden dimension	64, 128, 256	128
Branch embedding dimension	32, 64, 128	64
GLGA subgraph ratio	0.1-0.9	0.3
HSIC regularization weight	0, 0.1, 0.3, 0.5, 0.7, 1.0	0.5

Table S4. Sensitivity analysis of the GLGA subgraph selection ratio.

Ratio	Accuracy	Sensitivity	Specificity	MCC	AUC
0.1	0.842	0.829	0.854	0.683	0.910
0.2	0.842	0.842	0.842	0.683	0.915
0.3	0.860	0.862	0.858	0.720	0.918
0.4	0.837	0.837	0.837	0.675	0.915
0.5	0.842	0.842	0.842	0.683	0.915
0.6	0.829	0.821	0.837	0.659	0.907
0.7	0.833	0.842	0.825	0.667	0.913
0.8	0.829	0.837	0.821	0.659	0.910
0.9	0.833	0.829	0.837	0.667	0.911

Table S5. Performance comparison between GLGA and alternative graph aggregation methods. The best performance in each metric is highlighted in bold.

Pooling strategy	Accuracy	Sensitivity	Specificity	MCC	AUC
Mean pooling	0.825	0.858	0.793	0.652	0.912
Max pooling	0.823	0.817	0.829	0.646	0.908
Global attention pooling	0.848	0.846	0.850	0.695	0.916
TopKPool	0.831	0.842	0.821	0.663	0.913
SAGPool	0.837	0.846	0.829	0.675	0.904
Graph Transformer	0.842	0.837	0.846	0.683	0.915
GLGA	0.860	0.862	0.858	0.720	0.918

Table S6. Performance comparison of different Fusion strategies. The best performance in each metric is highlighted in bold.

Fusion strategy	Accuracy	Sensitivity	Specificity	MCC	AUC
Concatenation	0.829	0.821	0.837	0.659	0.908
Late fusion	0.842	0.846	0.837	0.683	0.909
Gated fusion	0.831	0.813	0.850	0.663	0.910
Bilinear fusion	0.823	0.817	0.829	0.646	0.908
Cross-attention fusion	0.860	0.862	0.858	0.720	0.918

Table S7. Performance comparison with machine learning baseline models. The best performance in each metric is highlighted in bold.

Model	Accuracy	Sensitivity	Specificity	MCC	AUC
RandomForest	0.839	0.772	0.907	0.685	0.906
ExtraTrees	0.829	0.772	0.886	0.663	0.902
GradientBoosting	0.817	0.776	0.858	0.636	0.911
AdaBoost	0.793	0.772	0.813	0.586	0.884
KNeighbors	0.762	0.756	0.768	0.524	0.829
DeepCPP	0.860	0.862	0.858	0.720	0.918

Table S8. Performance comparison with deep learning baseline models. The best performance in each metric is highlighted in bold.

Model	Accuracy	Sensitivity	Specificity	MCC	AUC
CNN	0.807	0.768	0.846	0.616	0.884
TCN	0.823	0.764	0.882	0.651	0.885
Transformer	0.811	0.732	0.890	0.630	0.872
MLP	0.793	0.776	0.809	0.586	0.867
DeepCPP	0.860	0.862	0.858	0.720	0.918

Supplementary Figures

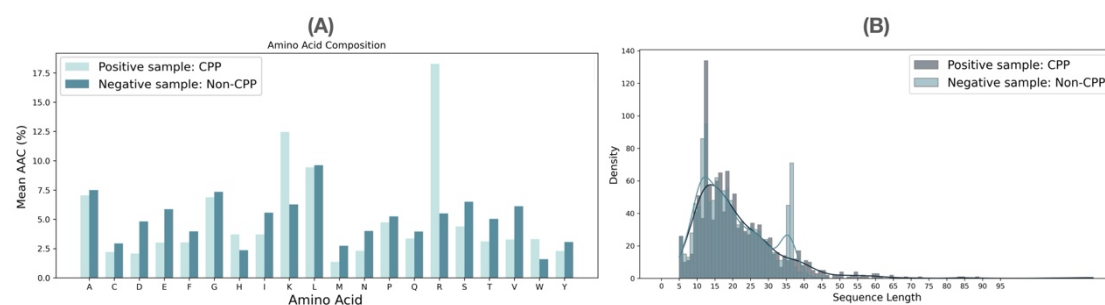


Figure S1. Statistics for dataset. (A) Mean AAC of positive samples (CPP) and negative samples (Non-CPP). (B) Length distribution of positive samples (CPP) and negative samples (Non-CPP).

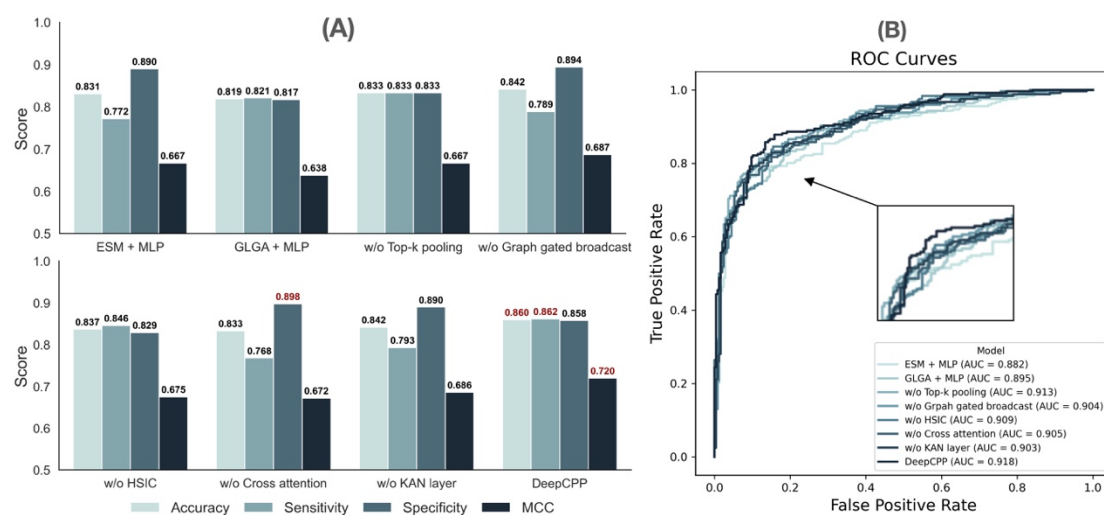


Figure S2. Ablation study of DeepCPP. (A) Performance comparison of model variants: ESM+MLP, GLGA+MLP, w/o Top-k pooling, w/o Graph gated broadcast, w/o HSIC, w/o Cross-attention, w/o KAN layer. (B) ROC curves for the same variants.